RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME ..... SIVARAM G .....

ACADEMIC YEAR 2026-2027 SEMESTER ..... V .....BRANCH..... CSD .....

UNIVERSITY REGISTER No. 211624170105 4

Certified that this is the bonafide record of work done by the above student in the GRAPHICS AND MULTIMEDIA ..... Laboratory during the year 20 26 - 20 27

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....03/11/2026.....

Internal Examiner **External Examiner**

G. SIVARAM
241701054
CSD-III

EX.NO	DATE	TITLE	TR.SIGN
1	21/7/2026	Bresenham's Line Drawing Algorithm for |M| < 1	
2	28/7/2026	Bresenham's Circle Algorithm	
3	28/7/2026	Bresenham's Ellipse Drawing Algorithm	
4	4/8/2026	DDA Line Drawing Algorithm	
5	4/8/2026	Monitor using Bresenham's Algorithm	
6	11/8/2026	Olympic Logo using Bresenham's Algorithm	
7	11/8/2026	Rectangle using Bresenham's Algorithm	
8	11/8/2026	Butterfly using Bresenham's Algorithm	
9	25/8/2026	Traffic signal using Bresenham's Algorithm	
10	25/8/2026	Smiling Face using Bresenham's Algorithm	
11	8/9/2026	Flood Fill Algorithm	
12	15/9/2026	Basic 2D Transformations	
13	22/9/2026	Polygon Clipping	
14	29/9/2026	Basic 3D Transformations	
15	06/10/2026	Simple Multimedia Application	
16	06/10/2026	Basic Chat Application	
17	13/10/2026	Create & render 3D Objects	

2/7/28

EXPERIMENT - 1

BRESENHAM'S LINE DRAWING ALGORITHM FOR |M|<1

AIM:

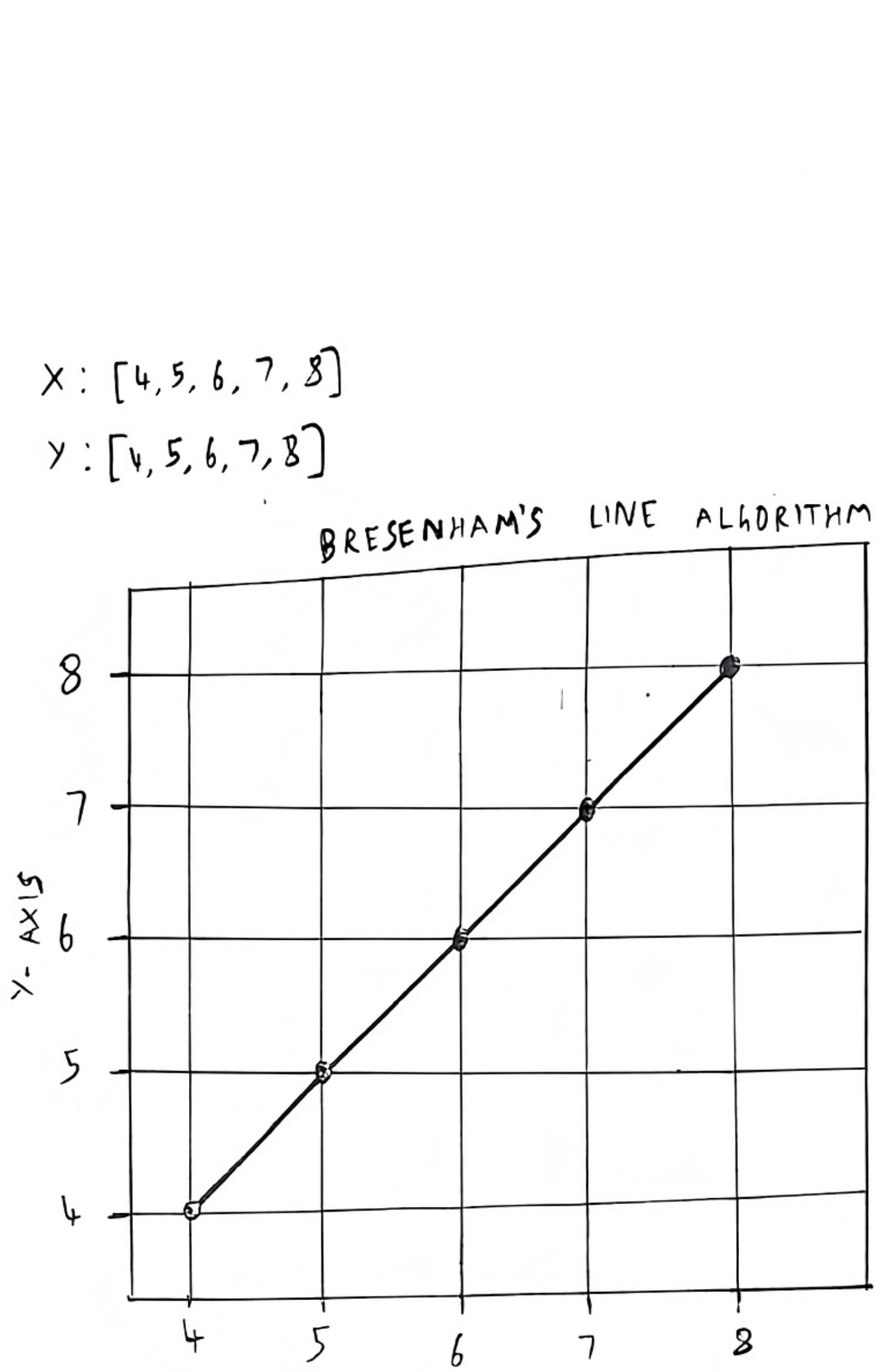
To implement Bresenham's Line Drawing Algorithm for |M|<1 using python programming.

PROGRAM:

```
import matplotlib.pyplot as plt
def bresenham_line(x1, y1, x2, y2):
    points = []
    dx = x2-x1
    dy = y2-y1
    p = 2*dy-dx
    x = x1
    y = y1
    points.append((x, y))
    while x<x2:
        x = x+1
        if p<0:
            p = p+2*dy
        else:
            y = y+1
            p = p+2*dy-2*dx
            points.append((x,y))
    return points
x1 = int(input("Enter x1:"))
y1 = int(input("Enter y1:"))
x2 = int(input("Enter x2:"))
y2 = int(input("Enter y2:"))
points = bresenham_line(x1, y1, x2, y2)
print("Points on lines:")
```

X : [4, 5, 6, 7, 8]

Y : [4, 5, 6, 7, 8]



O/P verified

21/7/20

```
for (x, y in Bip (x_points, y_points):
    print (f"({x}, {y})")
plt.figure (figsize) = (6, 6)
plt.plot (x, y, marker = 'S', linestyle = '-', color = 'blue')
plt.title = ('-', color = 'blue')
algorithm (m < 1)")
plt.xlabel ("x_axis")
plt ylabel ("y_axis")
plt.grid (True)
plt.axis ("equal")
plt show ()
```

RESULT:

The program was executed successfully, for input where slope $|m| < 1$

28/7/26

EXPERIMENT-02

BRESENHAM'S CIRCLE ALGORITHM

AIM:

To implement Bresenham's circle algorithm in python and generate the points of circle in coordinates

PROGRAM:

```
import matlatlib.pyplot as ptt
def bresenham_circle(xc,yc,r):
    points = set()
    xc = 0
    y = r
    d = 3-2*r
    def plot_sym_points(x_coord, y_coord):
        points.add((xc+x_coord, yc+y_coord))
        points.add((xc+x_coord, yc+y_coord))
        points.add((xc+x_coord, yc+y_coord))
        points.add((xc+x_coord, yc+y_coord))
        points.add((xc-y_coord, yc+x_coord))
        points.add((xc+y_coord, yc-x_coord))
        points.add((xc-y_coord, yc-x_coord))
        while y>=x:
            x+=1
            if d>0:
                y-=1
                d = d+4*x+6
            else:
                d = d+4*x+6
                plot_symmetric_points(x,y)
xc = int(input("Enter x coordinate:"))
yc = int(input("Enter y coordinate:"))
r = int(input("Enter radius:"))
```

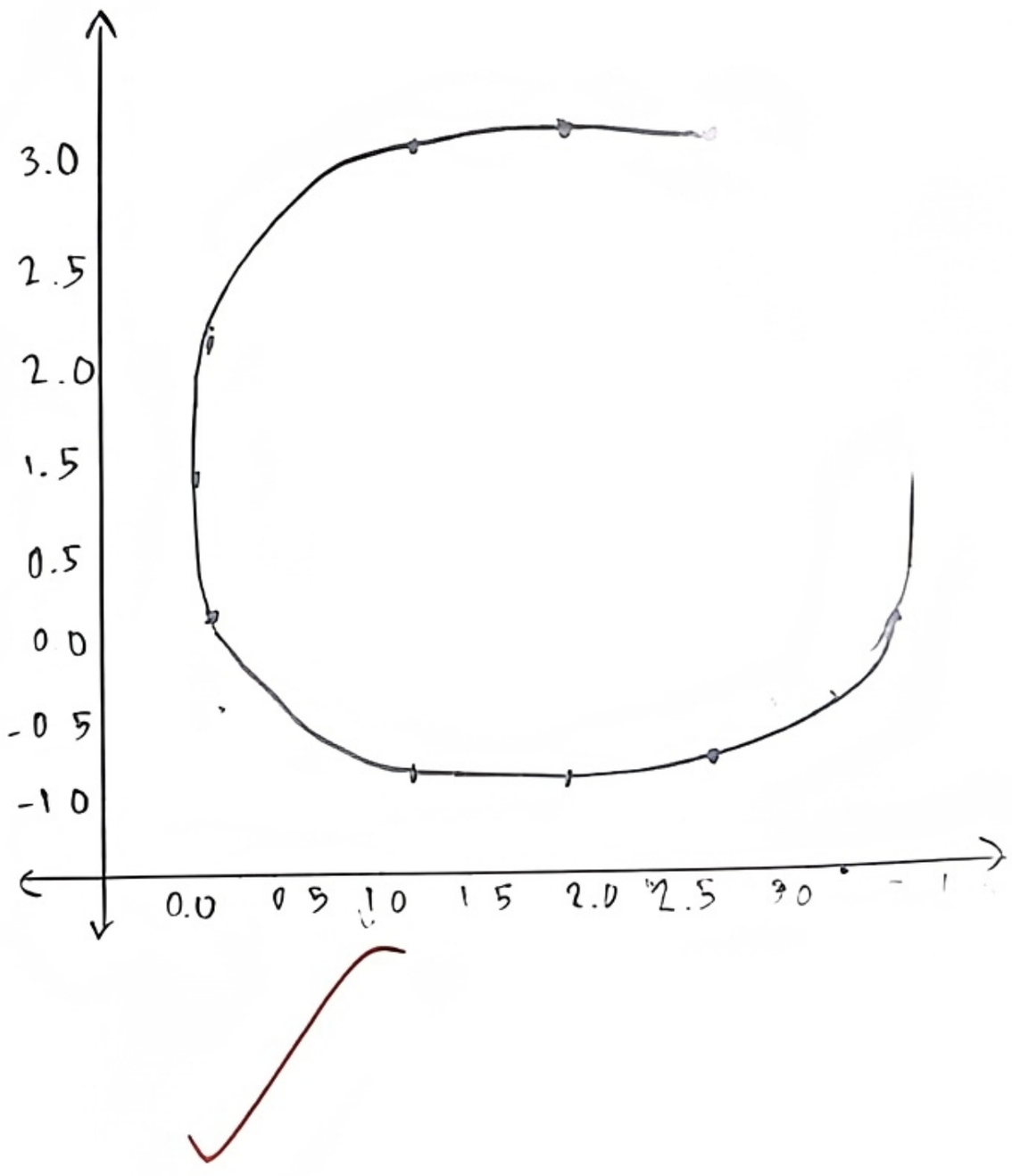
OUTPUT:

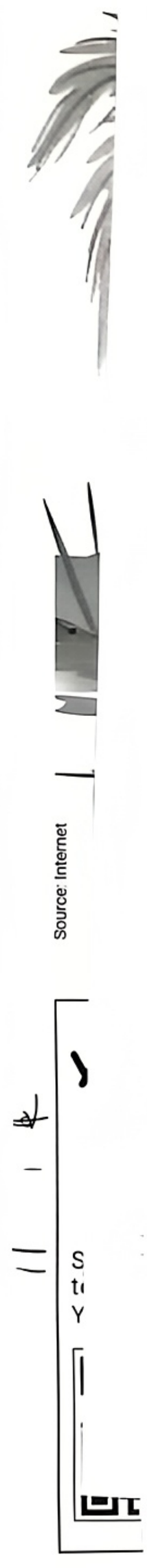
X - Co-ord of centre (x_c) : 2

Y - Co-ord of Center (y_c) : 1

Enter radius : 2

Points (0,0), (0,1), (0,2), (1,-1), (1,3), (2,-1),
(2,3), (3,-1), (3,3), (4,0), (4,1), (4,2)





Source: Internet

```
for p in circle_points:
    print(p)
x_coords = [p[0] for p in circle_points]
y_coords = [p[1] for p in circle_points]
plt.figure(figsize=(8,8))
plt.title("Bresenham's Circle Drawing Algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.show()
```

28/7/24

RESULT:

The Bresenham Circle Algorithm was implemented, program generated the intermediate pixel coordinates between is displayed

8/7/26

EXPERIMENT-3

BRESENHAM'S ELLIPSE DRAWING ALGORITHM

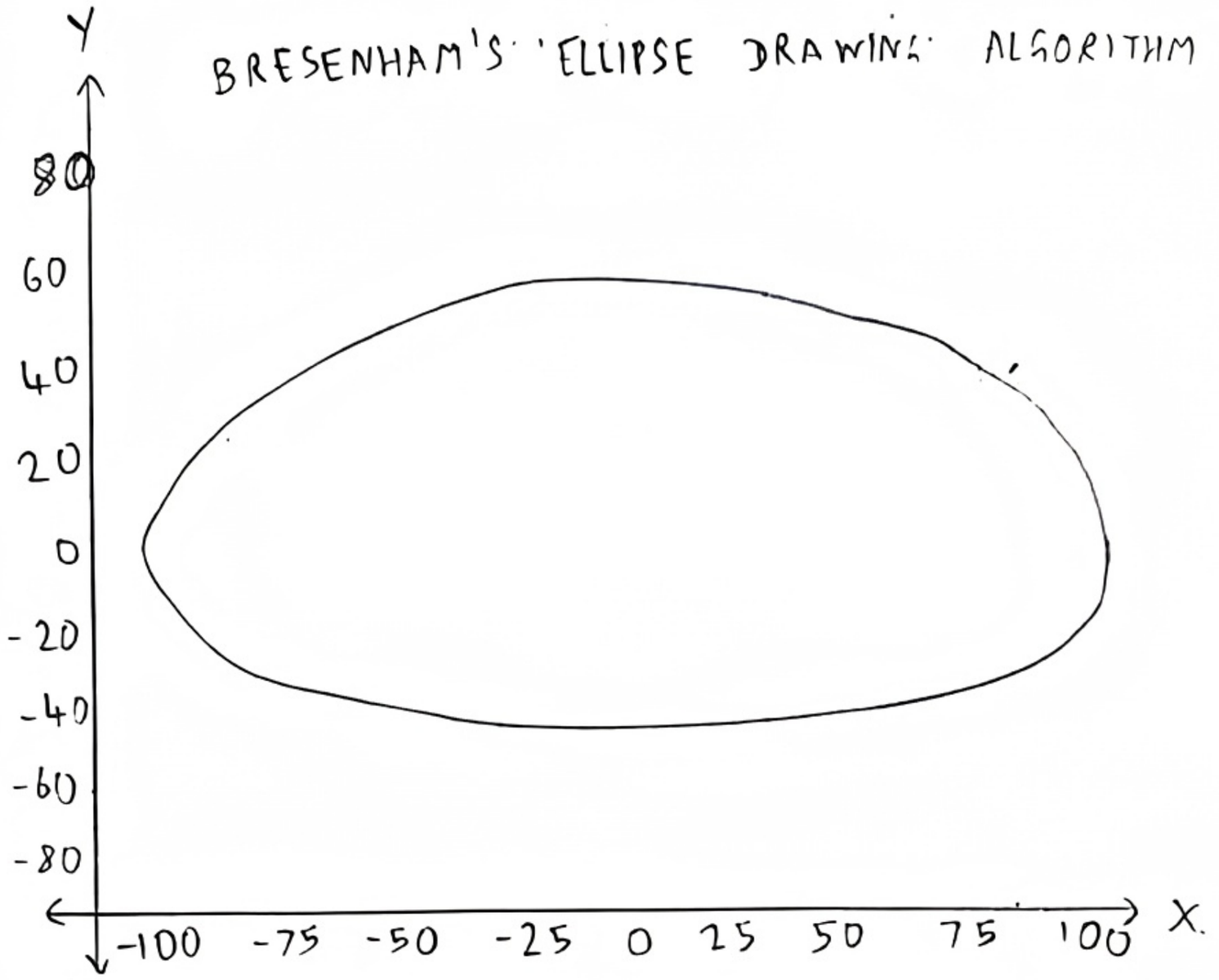
AIM:

To implement Bresenham's line drawing algorithm for eclipse using python programming

PROGRAM:

```
import matplotlib.pyplot as plt
def bresenham_ellipse(xc, yc, rx, ry):
    x = 0
    y = ry
    rx2 = rx * rx
    ry2 = ry * ry
    p1 = ry2 - (rx2 * ry) + (0.25 * rx2)
    points = []
    while (2 * ry) < (2 * rx):
        points.append((x, y))
        if p1 < 0:
            x = x + 1
        else:
            x = x + 1
            y = y - 1
            p1 = p1 + (2 * ry2 * x) - (2 * rx2 * y) + ry2
    while y >= 0:
        points.append((x, y))
        if p2 > 0:
            y = y - 1
            p2 = p2 - (2 * 2rx2 * y) + rx2
        else:
            x = x + 1
            y = y - 1
            p2 = p2 + (2 * ry2 * x) - \(2 * rx2 * y) + rx2
    for x, y in points:
        ellipse_points.append((xc + x, yc + y))
```

BRESENHAM'S ELLIPSE DRAWING ALGORITHM





Source: Internet

```
            ellipse_points.append((xc-x, yc+y))
            ellipse_points.append((xc+x, yc-y))
            ellipse_points.append((xc-x, yc-y))
        return ellipse_points

xc=0
yc=0
rx=100
ry=50
x_values=[p[0] for points in P]
y_values=[p[1] for p in points]
plt.scatter(x_values, y_values, s=2)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Bresenhom's Ellipse Drawing Algorithm")
plt.axis("equal")
plt.grid()
plt.show()
```

28/7/26

RESULT:

Hence implemented Bresenham's line drawing algorithm for ellipse using python programming.

18/26

EXPERIMENT-04

DIGITAL DIFFERENTIAL (LINE DRAWING) ANALYZER ALGORITHM

AIM:

To implement DDA Algorithm for drawing line using python programming

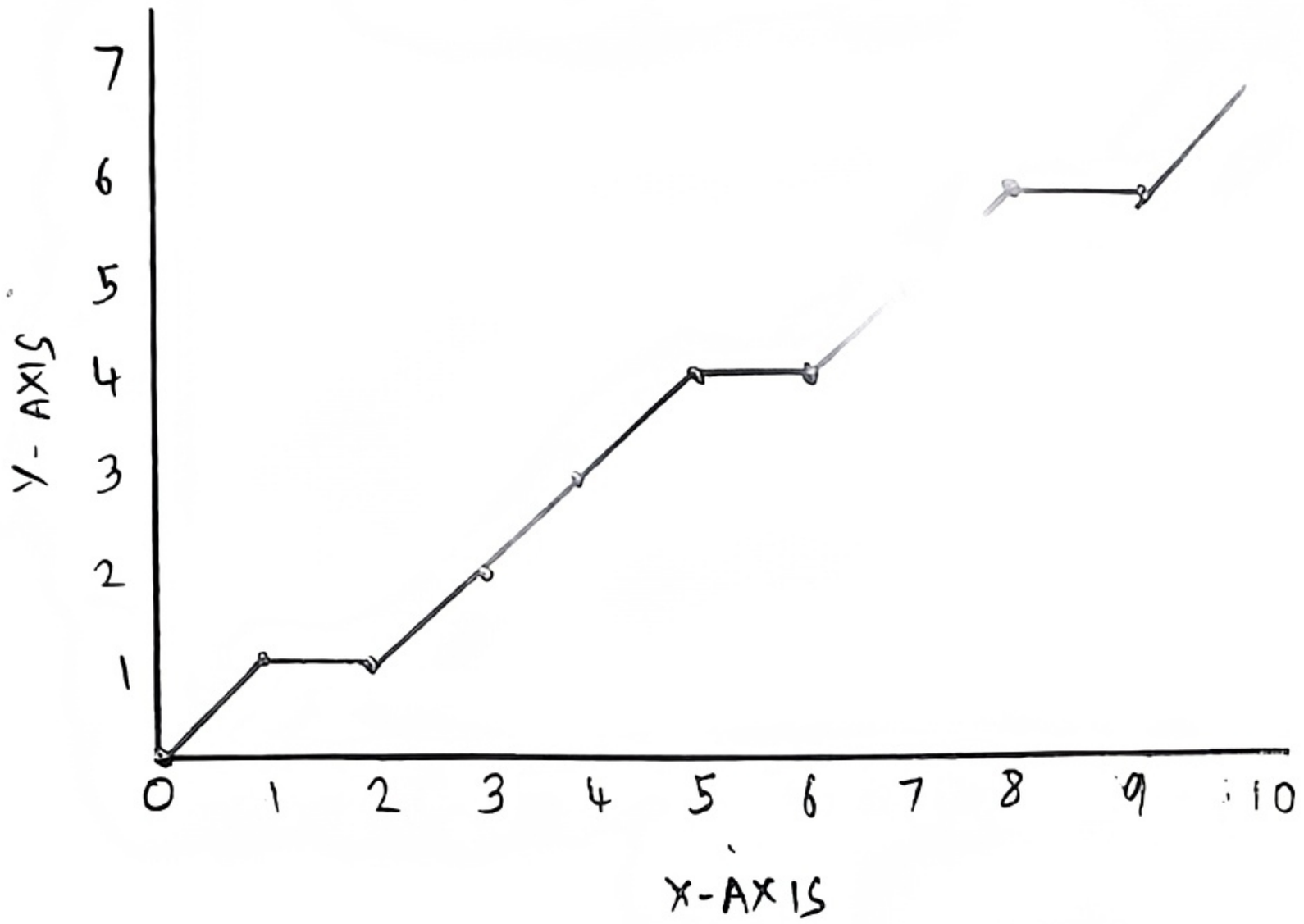
PROGRAM:

```
import matplotlib.pyplot as plt
def dda_line_drawing(x0, y0, x1, y1):
    dx = x1 - x0
    dy = y1 - y0
    steps = abs(dx) if abs(dx) > abs(dy)
                    else abs(dy)

    x = x0
    y = y0
    points = []
    points.append((round(x), round(y)))
    for _ in range(steps):
        x += x_increment
        y += y_increment
    return points

x0, y0 = 0, 0
x1, y1 = 10, 7
line_points = dda_line_drawing(x0, y0, x1, y1)
plt.figure(figsize=(8, 6))
plt.plot(x_coord, y_coord, 'ko-')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
```

DDA Line Drawing from (0,0) to (10,7)



Points Generated by DDA Algorithm

[(0,0), (1,1), (1,2), (3,2), (4,3), (5,4), (6,4),
(7,5), (8,6), (9,6), (10,7)]

Source: Internet

```
plt.xticks(range(min(x_coord), max(x_coord)+1))
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
print(f"\nPoints generated by DDA Algorithm:
{line_points}")
```

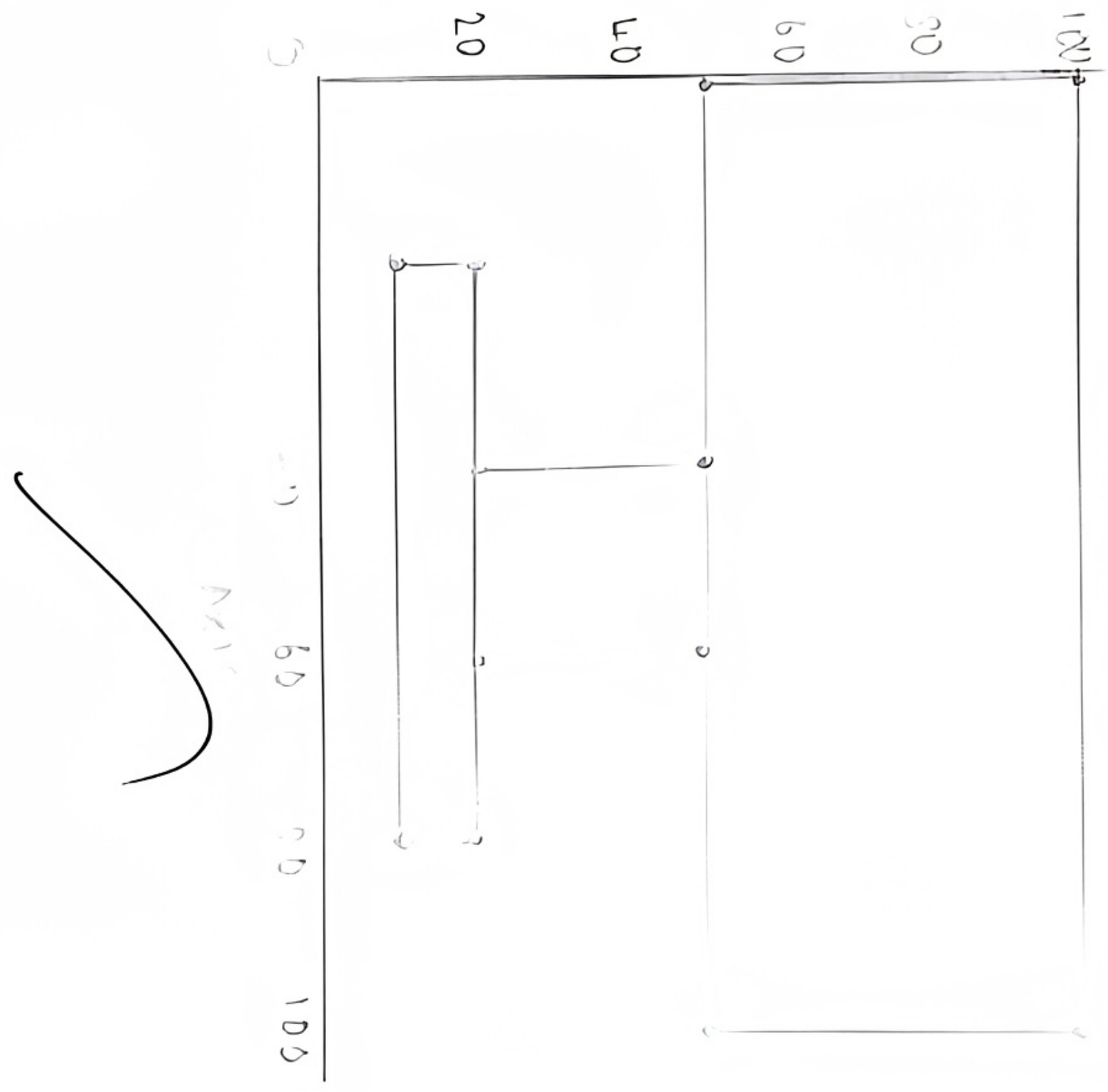
4/8/26

RESULT:

Hence, implemented DDA's Line Drawing Algorithm for the points generated using python programming.

Source: Internet

Monitor Drawn with Bresenham Line Algorithm



8/26

EXPERIMENT-05

MONITOR DRAWN USING BRESENHAM'S LINE ALGORITHM

AIM:

To implement Bresenham's Line Algorithm to draw monitor using python programming.

PROGRAM:

```
fig, axc = plt.subplots (figsize = (8, 8))
axc.set_aspect ('equal', adjustable = 'box')
axc.set_xlabel ("X-Axis")
axc.set_ylabel ('Y-Axis')
ax.set_title ('Monitor drawn using Bresenham's
                                Line Algorithm')
axc grid (True)
screen_width = 100
screen_height = 50
screen_width = 20
stand_height = 30
base_width = 60
base_height = 10
xc_offset = 0
y_offset = 50
min_x = x_offset - 10
ax.set_xlim (min_x, max_x)
plt.show()
```

RESULT:

Hence, successfully had the Monitor output for Bresenham's Line Algorithm

18/26

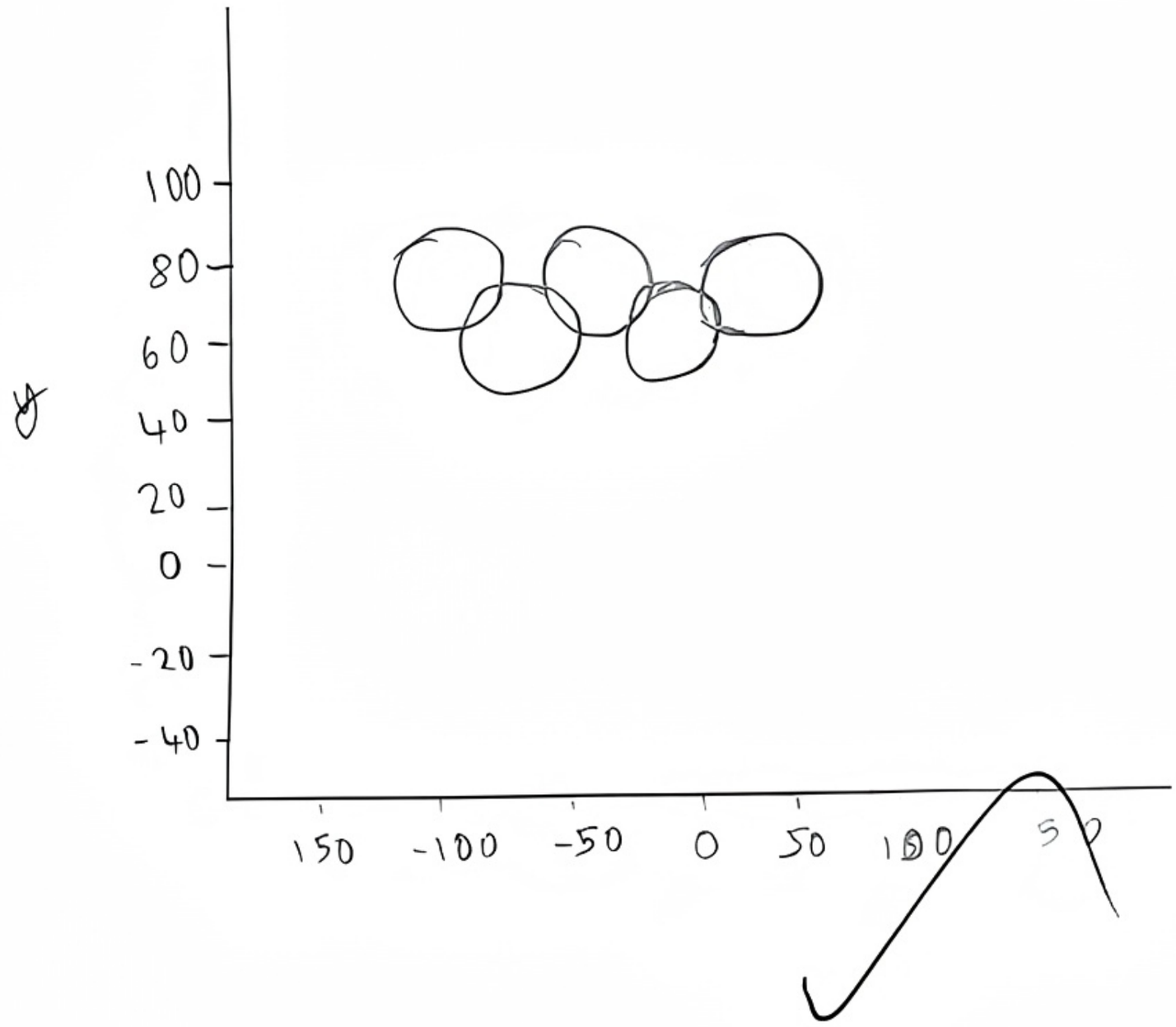
EXPERIMENT-06

AIM:

To draw olympic logo using Bresenhom's algorithm

PROGRAM:

```
import matplotlib.pyplot as plt
def midpoint_ellipse(xc, yc, rx, ry):
    points = []
    x = 0x
    y = ry
    rx2 = rx * rx
    ry2 = ry * ry
    dx = 2 * ry2 * x
    dy = 2 * rx2 * y
    p1 = ry**2 - (rx * 2 * ry) + (0.25 * rx * 2)
    if p1 < 0:
        x += 1
        dx = 2 * ry**2 * x
        p1 += dx + ry2
    while y >= 0:
        points.extend([cx(+x, yc+y)(xc-x, yc+y),
                       (xc+y, yc-y), (xc-x, yc-y)])
        if p2 > 0:
            y -= 1
            dy = 2 * ry * 2 * y
            p2 += rx * 2 - dy
        else:
            x += 1
            y -= 1
            dx = 2 * ry2 * x
            dy = 2 * rx * 2 * y
    return points
```

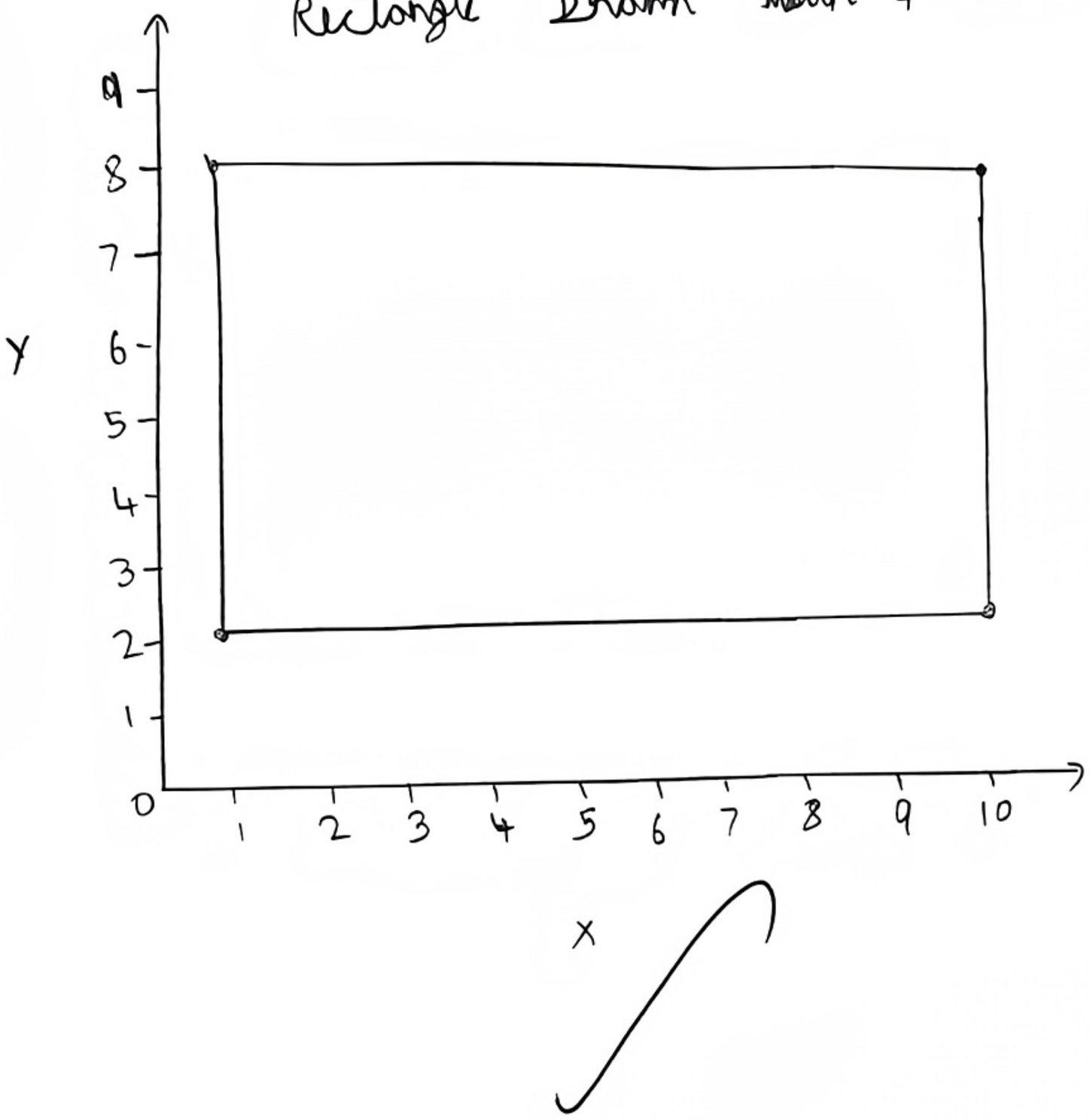

100
80
60
40
20
0
-20
-40
150
-100
-50
0
50
100
50

```
for xc, yc, color in range:
    pts = midpoint_ellipse (xc, yc, 40, 40)
    x = [p[0] for p in pts]
    y = [p[1] for p in pts]
    plt.axis ('off')
    plt.show()
```

RESULT:

Hence, successfully drawn olympic logo using ellipse was drawn.

Rectangle Drawn with 4 Vertices



11/8/24

EXPERIMENT-07

AIM:

To draw rectangle using Bresenham's Algorithm

PROGRAM:

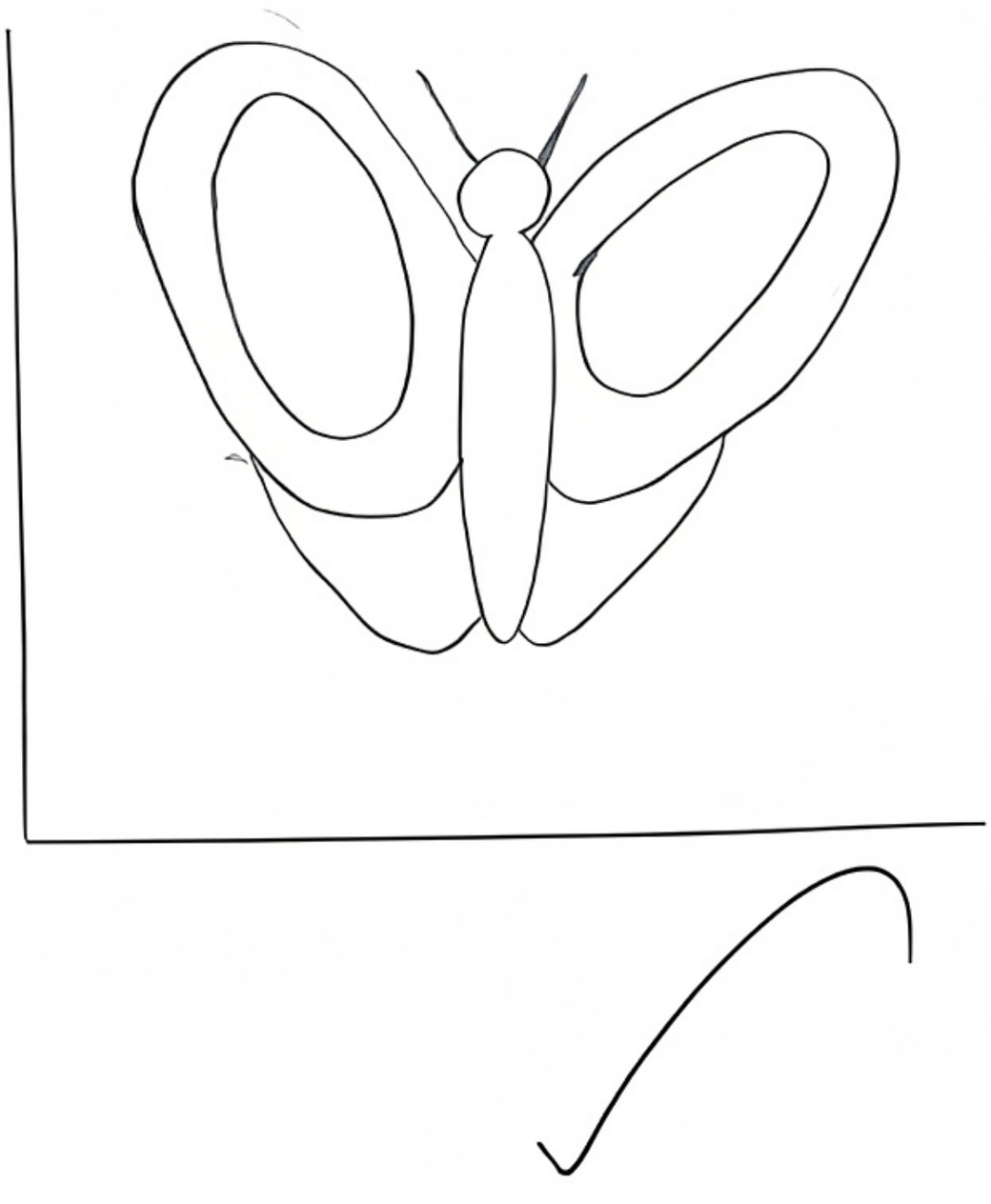
```
import matplotlib.pyplot as plt

x = 2
y = 2
width = 8
height = 5
top_left = (x, y)
top_right = (x + width, y)
px = [top_left[0], top_right[0], bottom_right[0],
      bottom_left[0], top_left[0]]
py = [top_left[1], top_right[1], bottom_right[1],
      bottom_left[1], top_left[1]]

plt.grid(True)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Rectangle with 4 Vertices")
plt.gca().invert_yaxis()
plt.show()
```

RESULT:

Hence, drawn using the Rectangle in Bresenham's algorithm successfully.



11/8/21

EXPERIMENT-08

AIM:

To draw a butterfly using Bresenham's Algorithm

PROGRAM:

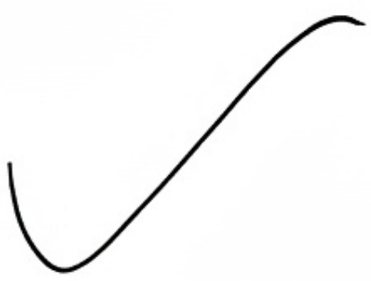
```
import matplotlib.pyplot as plt
from matplotlib.patches as Ellipse
fig, axc = plt.subplots (figsize = (6,6))
axc.add_patch (Ellipse ((-0.6, -0.4), 1.2, 1.6, alpha = 0.8))
axc.add_patch (Ellipse ((0.6, -0.4), 1.2, 1.6), angle = 40,
                                                alpha = 0.8))
axc.add_patch (Ellipse ((-0.9, 0.5), 1.8, 2.2), alpha = 0.9, angle = 35)
axc.add_patch (Ellipse ((0.9, 0.5), 1.8, 2.2), angle = -35,
                                                alpha = 0.9))

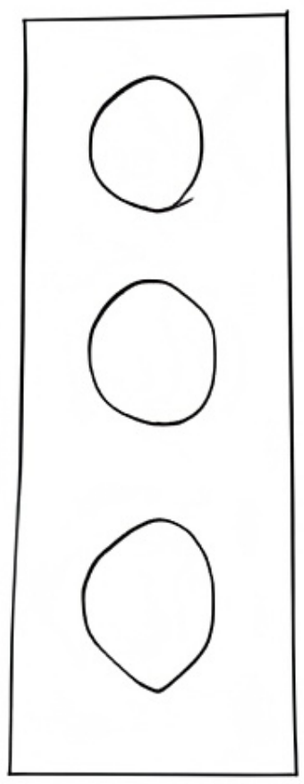
axc.plot ([-0.05, -0.3], [1.2, 1.7], color = 'saddlebrown'))
axc.plot ([0.05, 0.3], [1.2, 1.7], color = 'saddlebrown'))
axc.set_xlim (-2.5, 2.5)
axc.set_ylim (-2.0, 2.0)
axc.set_aspect ('equal')
axc.axis ('off')
plt.show ()
```

RESULT:

Hence, successfully drawn butterfly using Bresenham's algorithm







12/15/22

EXPERIMENT-09

AIM:

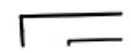
To draw traffic signal using Bresenham's Algorithm

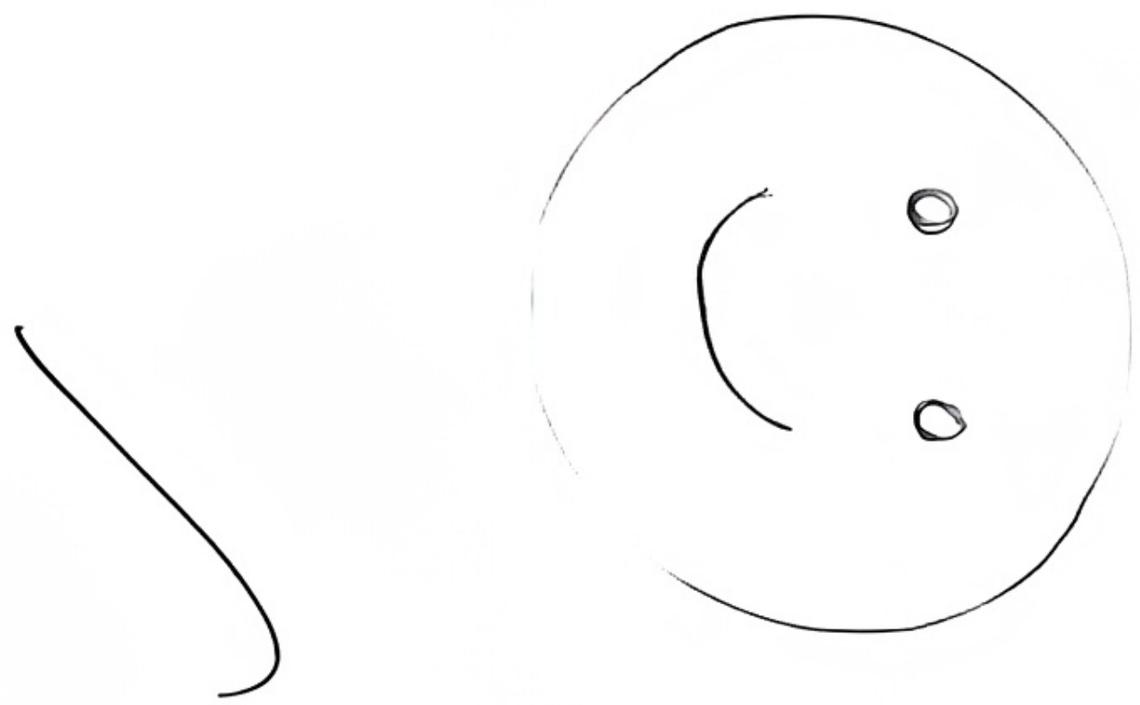
PROGRAM:

```
import matplotlib.pyplot as plt
import matplotlib.patches as patches
def draw_static_light(active_color):
    fig, ax = plt.subplots(figsize=(3,6))
    housing = patches.Rectangle((1,1),(3,7))
    ax.add_patch(housing)
    red_color = '#ff3333' if active_color=='red'
    green = '#ffcc00' if active_color == 'yellow'
    yellow='#33ff33' if active_color == 'green'
    ax.add_patch(red_circle)
    ax.add_patch(yellow_circle)
    ax.add_patch(green_circle)
    ax.set_xlim(0,5)
    ax.set_ylim(0,9)
    ax.axis('off')
    plt.show()
draw_static_light(active_color='red')
```

RESULT:

Hence, successfully drawn traffic signal using Bresenham's algorithm.





12/12/24

EXPERIMENT-10

AIM:

To draw smiling face using Bresenham's Algorithm.

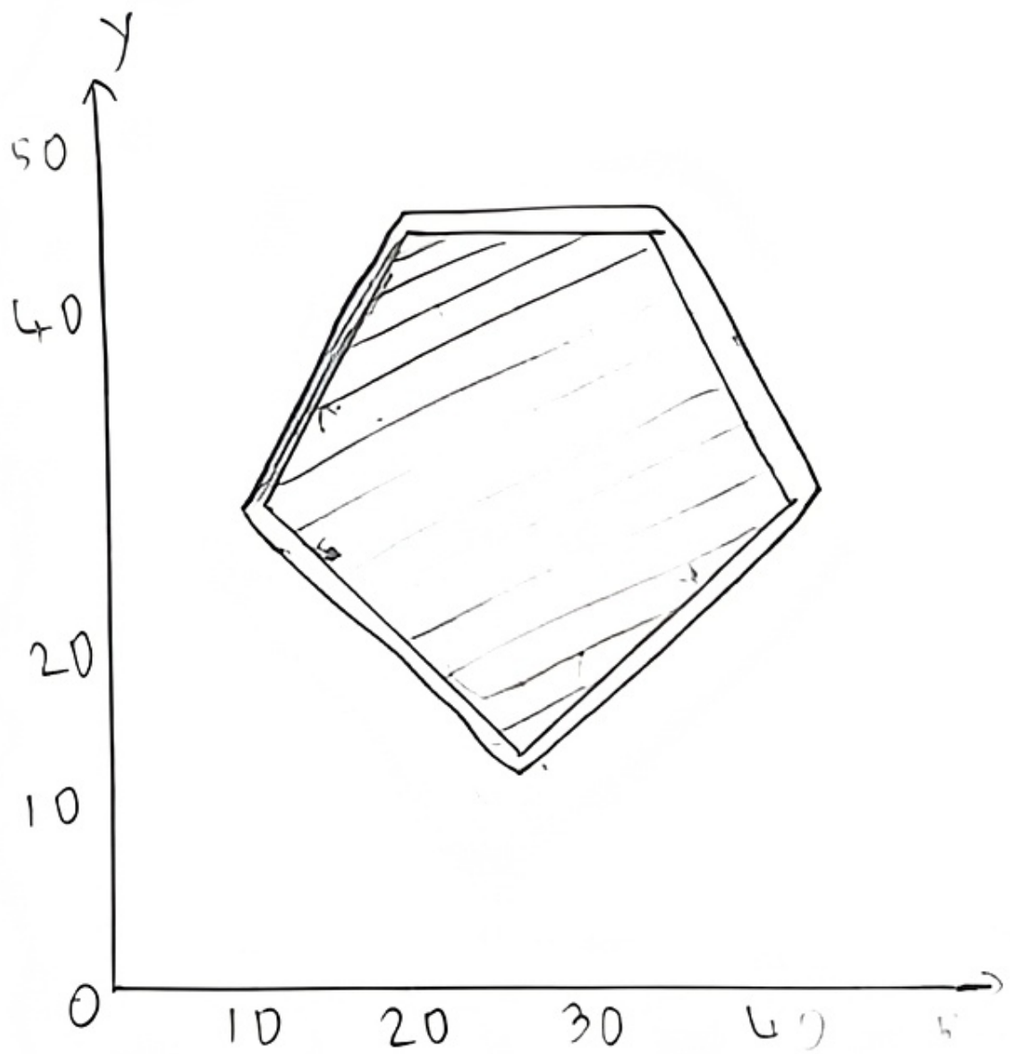
PROGRAM:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots (figsize = (4,4))
ax.add_patch (( 0,0), 1.0, color = 'yellow'))
 ax.add_patch [(-0.35, 0.35), (0.12), color=('black'))
ax.add_patch ((0.35, 0.35), 0.12, color = 'black'))
smile_X = [-0.4, -0.2, 0, 0.2, 0.4]
smile_Y = [-0.2, -0.4, -0.45, -0.4, -0.2]
ax.plot ( smile_X, smile_Y, color = 'black', lw = 3)
ax.set_xlim (-1.2, 1.2)
ax.set_ylim (-1.2, 1.2)
ax.axis ("off")
plt.show ()
```

RESULT:

Hence, successfully drawn smiling face using Bresenham's algorithm.

Polygon Scan



5/9/26

EXPERIMENT - 11

AIM:

To implement scan conversion of polygon and use flood fill algorithm

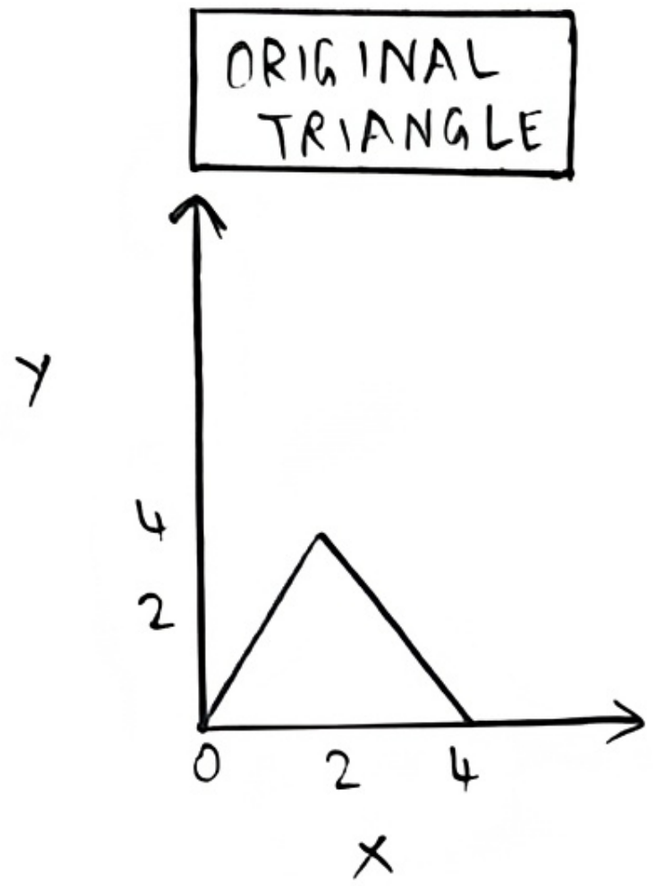
PROGRAM:

```
import matplotlib.pyplot as plt
def line(grid, x0, y0, x1, y1, color):
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    err = dx - dy
    while True:
        if x0 == x1 && y0 == y1:
            break
        e2 = 2 * err
        if e2 > -dx:
            x0 += sx
        y0 += sy
def flood_fill(grid, start_x, fill_color, target_color):
    rows = len(grid)
    cols = len(grid[0])
    queue = deque([(start_x, start_y)])
    while queue:
        cx, cy = queue.popleft()
        if grid[ny][nx] == target_color:
            grid[ny][nx] == fill_color
            queue.append((nx, ny))
def main():
    width, height = 50, 50
    vertices = [(25, 5), (45, 20), (35, 40), (15, 40), (5, 20)]
    edge_color = 1
    fill_color = 2
    fr = 0
    num_vertices = len(vertices)
    for i in range(num_vertices):
        line(grid, x0, y0, x1, y1, edge_color)
    flood_fill(grid, fill_color, fr)
    plt.figure(6,6)
    plt.show()
plt.__name__ == "__main__":
    main()
```

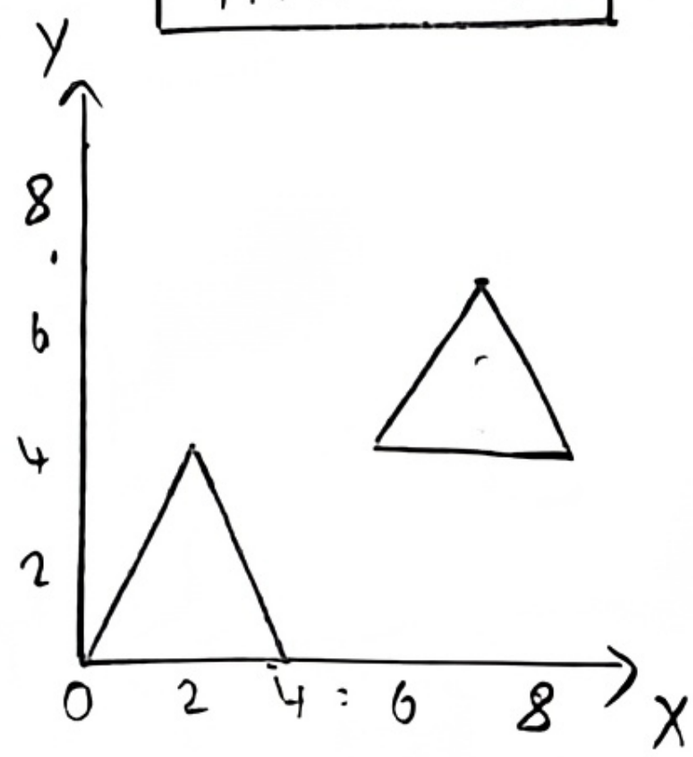
RESULT:

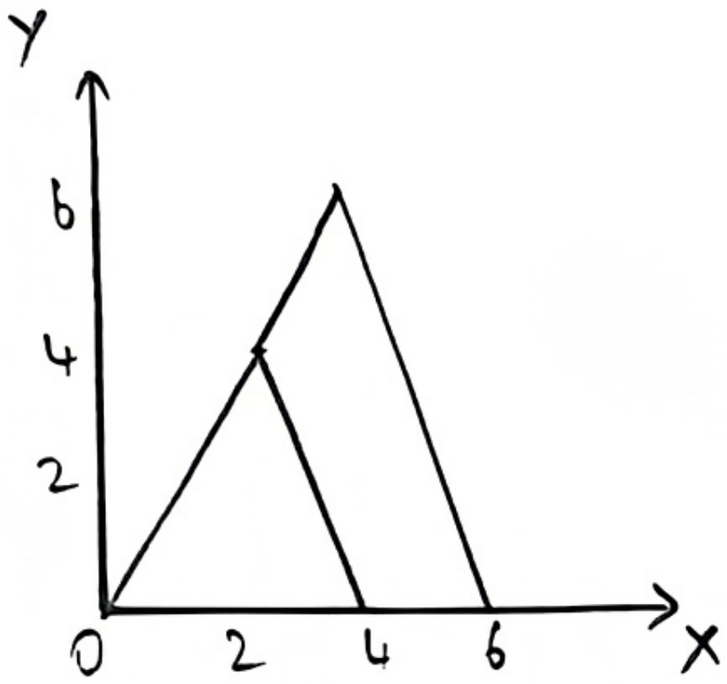
Hence, successfully implemented scan conversion of polygon using flood fill algorithm.

ORIGINAL TRIANGLE

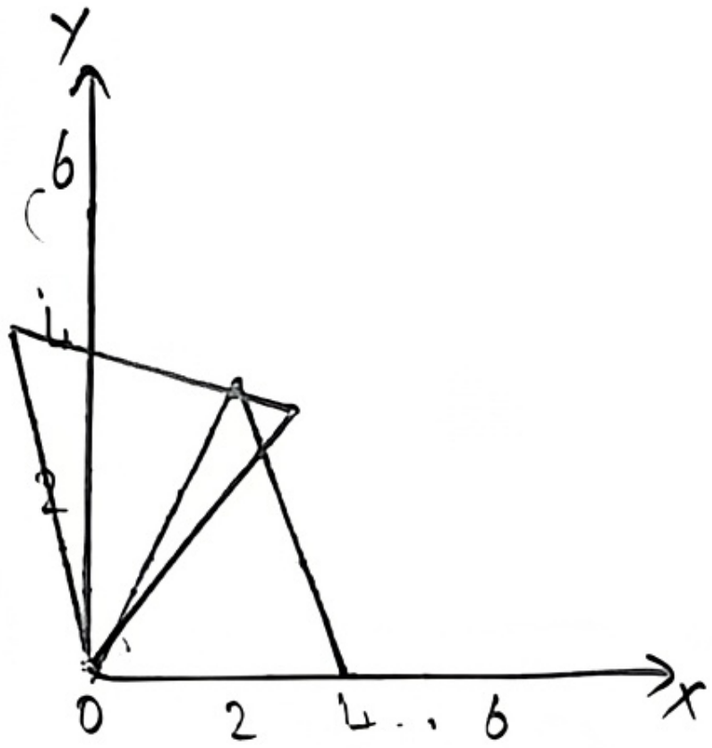


TRANSLATION





SCALING



ROTATION

5/9/26

EXPERIMENT-12

AIM:

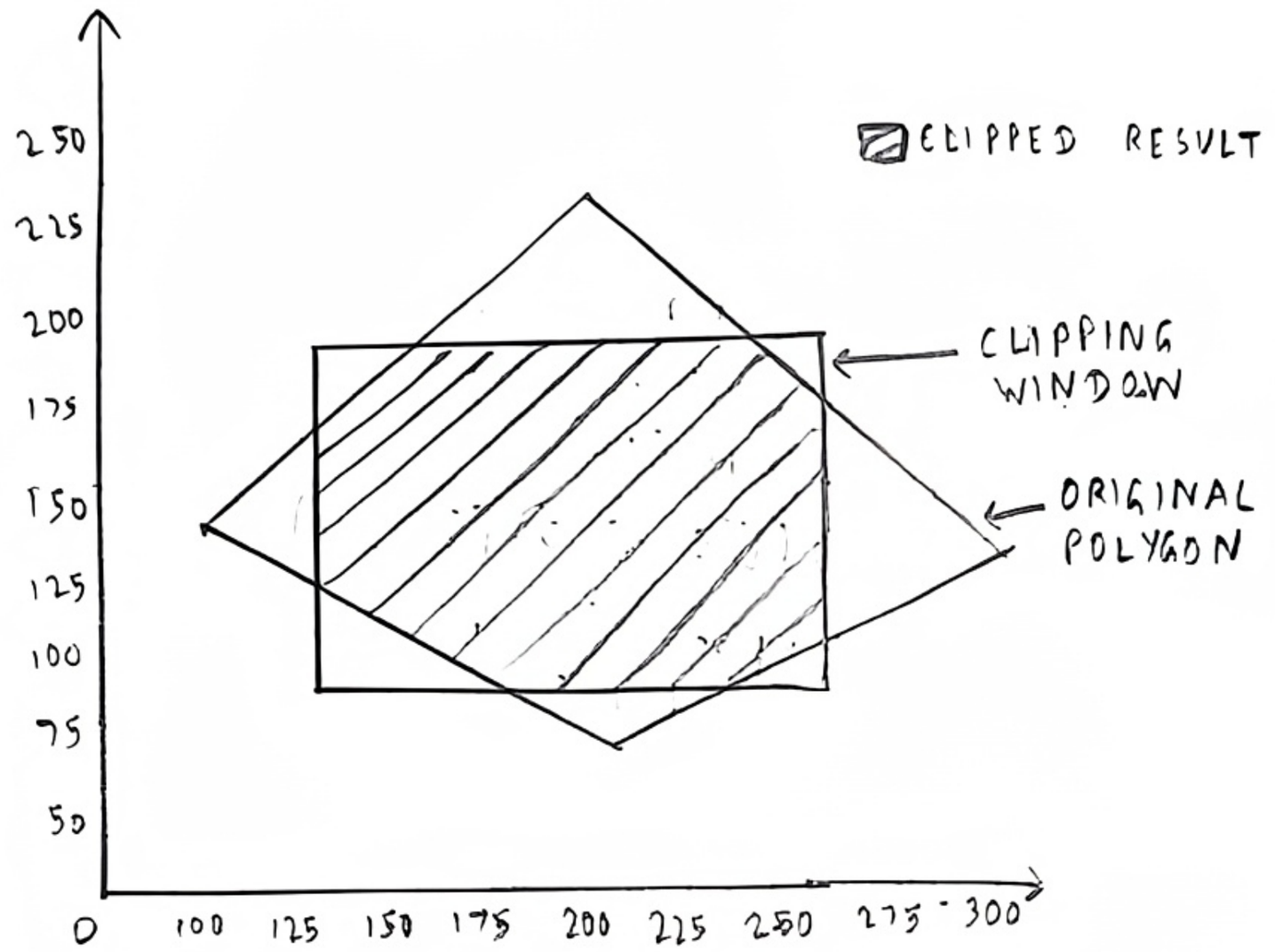
To implement basic 2D transformations using 2D shapes.

PROGRAM:

```
import matplotlib.pyplot as plt
import numpy as np
triangle = np.array([[0,4,2],[0,0,3],[1,1,1]])
def plot(ax, pts, color, alpha = 0.4)
    ax.plot(x, y, color = color, marker = "o")
    ax.fill(x, y, color = color, alpha = alpha)
    T = np.array([[1,0,4],[0,1,3],[0,0,1]])
    S = np.array([[1.5,0,0],[0,1.5,0],[0,0,1]])
    rad = np.radians(45)
    R = np.array([[c,-s,0],[s,c,0],[0,0,1]])
    t_triangle = np.dot(T, triangle)
    s_triangle = np.dot(S, triangle)
    R_triangle = np.dot(R, triangle)
def format_subplot(ax, title):
    ax.set_xlim(-6,8)
    ax.set_ylim(-6,8)
    ax.grid(True, linestyle = "--", alpha = 0.5)
    ax.set_title(title)
plot_triangle(ax[0,0], triangle, "blue", "Original", alpha = 0.3)
format_subplot(ax[0,0], "1. Original Triangle")
plot_triangle(ax[0,0], traingle, "grey", "Original", alpha = 0.2)
format_subplot(ax[0,1], "2. Translation")
plot_triangle(ax[1,0], triangle, "grey", "Original", alpha = 0.2)
format_subplot(ax[1,0], "3. Scaling")
plot_triangle(ax[1,1], triangle, "grey", "Original", alpha = 0.2)
format_subplot(ax[1,1], "4. Rotation")
plt.tight_layout()
plt.show()
```

RESULT:

Hence, successfully implemented basic 2D transformations using 2D shapes using python program.



CLIPPED VERTICES : [(150.0, 200.0), (250.0, 200.0), (280.0, 170.0), (280.0, 130.0), (230.0, 80.0), (170.0, 80.0), (120.0, 130.0), (120.0, 170.0)]

29/9/26

EXPERIMENT-13

AIM:

To implement polygon clipping using python programming

PROGRAM:

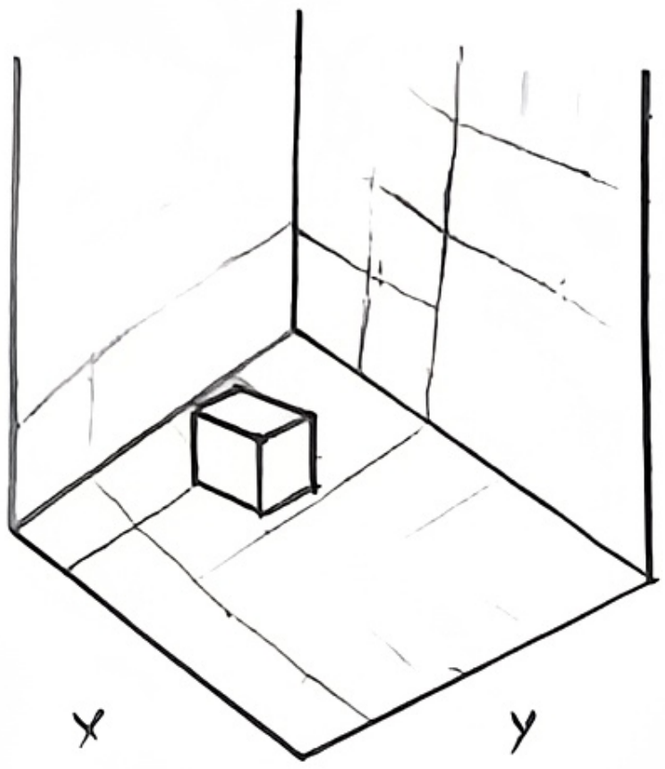
```
! pip install -q shapely matplotlib
import matplotlib.pyplot as plt
from shapely.geometry import Polygon
poly_subj = Polygon ( [(100,150), (200,250), (300,150)])
play_clip = Polygon ([(120,80), (120,200), (280,200)])
poly_result = ply_subj intersection ( poly_clip)
fig, asc = plt.subplots ( figsize = (6,6))
asc.plot (*poly_subj.exterior.xy, 'b--')
ax.plot (*poly_clip.exterior.xy, 'g:')
ax fill (*poly_result.exterior.xy, 'r', alpha = 0.4)
ax.set_title ('Sutherland-Hodgan Clipping')
asc.legend ()
asc.grid (True)
plt.show ()
print ("Clipped Vertices:", list (poly_result.exterior)[:-1])
```

29/9/26

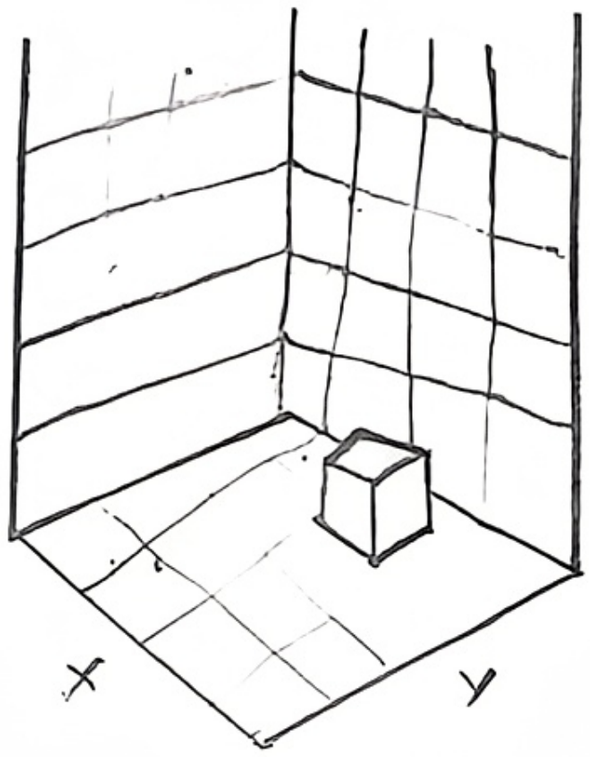
RESULT:

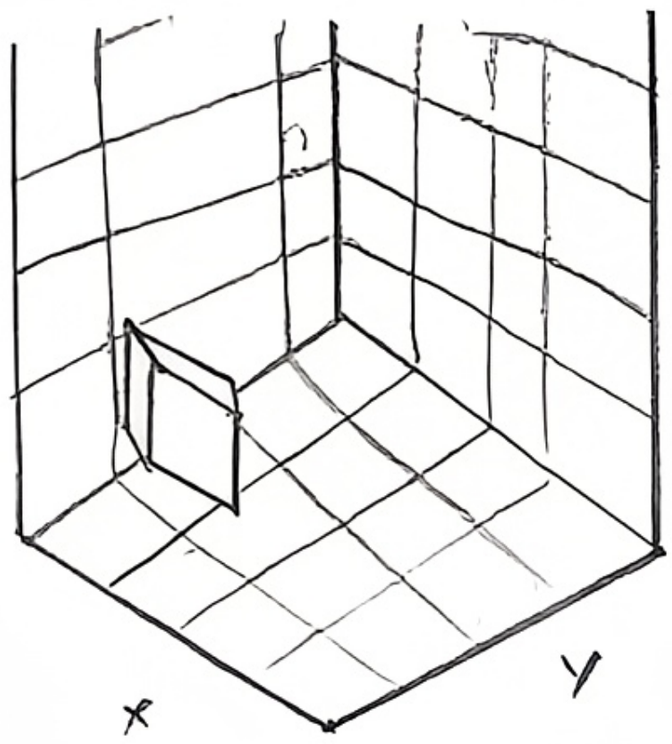
Hence, successfully implemented polygon clipping using python programming.

ORIGINAL CUBE

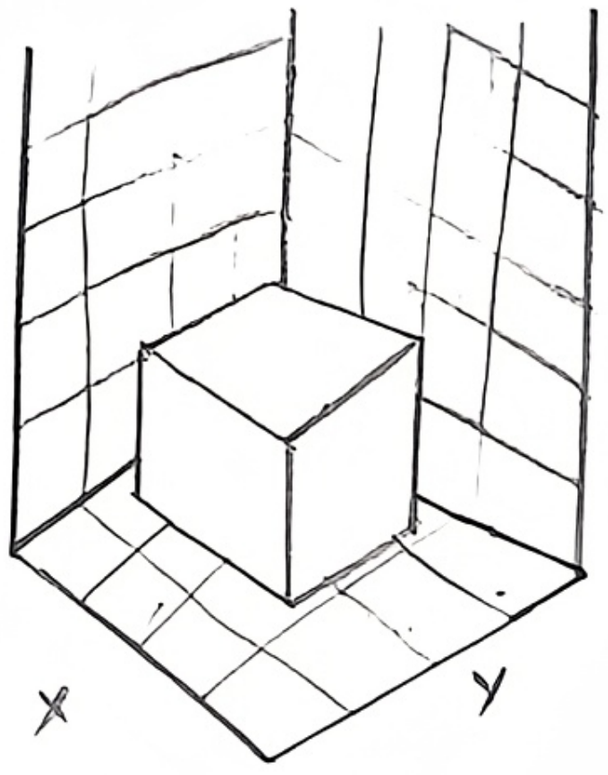


TRANSLATION





ROTATION 45°



SCALING 1.5X

20/4/26

EXPERIMENT-14

AIM: To implement basic 3D Transformations using 3D shapes.

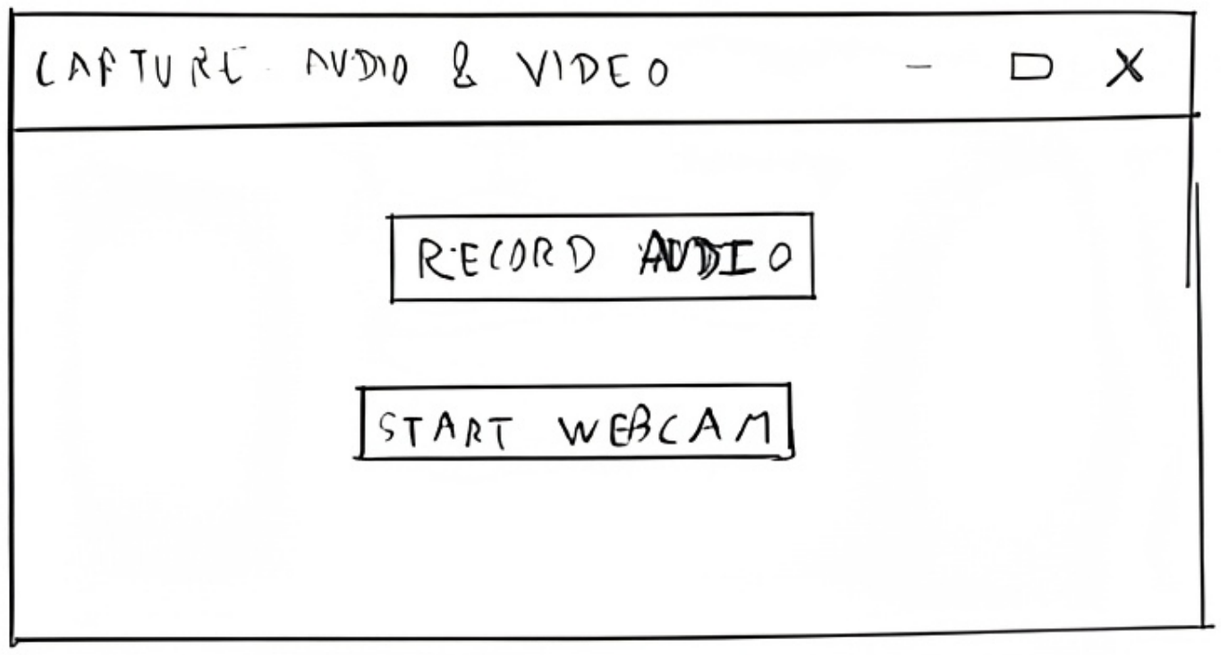
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
vertices = np.array([[0,0,0], [2,0,0], [2,2,0], [0,2,0], [0,0,2],
                     [2,0,2], [2,2,2], [0,2,2]], dtype=float)
faces = [[0,1,2,3], [4,5,6,7], [0,1,5,4], [2,3,7,6], [0,3,7,4], [1,2,6,5]]
translation = np.array([3,2,1])
angle = np.radians(45)
rotated = vertices @ rotation_matrix.T
scale_matrix = np.array([[1.5,0,0], [0,1.5,0], [0,0,1.5]])
scale = vertices @ scale_matrix.T
def draw_cube(ax, points, title)
    cube_faces = [[points[i] for i in face] for face in faces]
    ax.set_title(title)
    ax.set_xlim(-2,7)
    ax.set_ylim(-2,7)
    ax.set_zlim(-2,7)
    ax.set_xlabel("X")
    ax.set_ylabel("Y")
    ax.set_zlabel("Z")

fig = plt.figure(figsize=(15,4))
ax1 = fig.add_subplot(141, projection='3D')
draw_cube(ax1, vertices, "original Cube")
ax2 = fig.add_subplot(142, projection='3D')
draw_cube(ax2, translated, "Translation")
ax3 = fig.add_subplot(143, projection='3D')
draw_cube(ax3, rotated, "Rotation 45°")
ax4 = fig.add_subplot(144, projection='3D')
draw_cube(ax4, scaled, "Scaling 1.5x")
plt.tight_layout()
plt.show()
```

RESULT:

Hence, successfully implemented 3D Transformations using 3D shapes.


CAPTURE AUDIO & VIDEO
RECORD AUDIO
START WEBCAM

EXPERIMENT-15

AIM: To develop a simple multimedia application into unified user interface using Python

PROGRAM:

```
import tkinter as tk
import cv2
def play_audio():
    playsound ('sample_audio.mp3')
def play_video():
    cap = cv2.VideoCapture('sample_video.mp4')
    while cap.isOpened():
        ret, cap.read()
    cap.release()
def load_image()
    img = img.resize(300, 300)
    img_tk = ImageTk.Photo(img)
    panel_img = img_tk
window = tk.Tk()
window.title("Multimedia App")
panel = tk.Label(Window)
panel.pack()
btn_img.pack(pady = 5)
btn_img.pack(pady = 15)
btn_audio.pack(pady = 15)
btn_video.pack(pady = 5)
window.mainloop()
```

RESULT

Hence, successfully implemented simple multimedia application into unified user interface using Python.

EXPERIMENT - 16

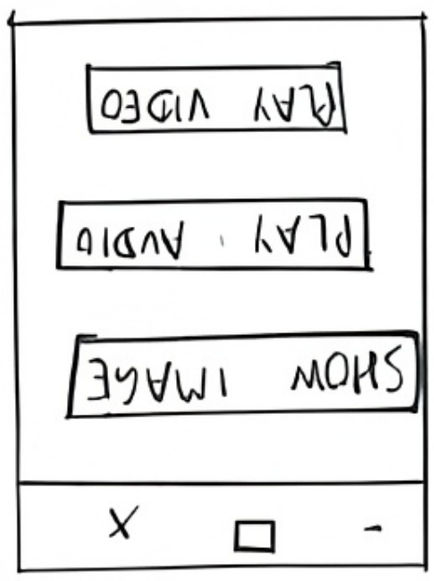
AIM: To develop a basic chat-like application that allows sending & receiving text, images and audio

PROCEDURE:

```
import tkinter as tk
from playsound import playsound
import threading
def send_text():
    msg = text_entry.get()
    if msg:
        chatbox.insert(tk.END, f"You: {msg}\n")
def send_img():
    if path:
        img = Image.open(path)
        img.thumbnail(100, 100)
        img_label = img_tk
        img.pack()
def send_audio():
    if path:
        chat_box.insert(tk.END)
window = tk.Tk()
window.title("Multimedia")
window.geometry("400x500")
chat_box.pack()
chat_frame.pack()
btn_text.pack(side=LEFT)
btn_image.pack(side=LEFT)
btn_audio.pack(side=LEFT)
window.mainloop()
```

RESULT:

Hence, successfully implemented to develop basic chat-like application that allows sending & receiving text, images and audio


SHOW IMAGE
PLAY AUDIO
PLAY VIDEO
-
X

EXPERIMENT-17

AIM:

To create & render 3D objects using polygon modelling

PROGRAM

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],[0,0,1],
                     [1,0,1],[1,1,1],[0,1,1]])

faces = [[vertices[i] for i in [0,1,2,3]]
         [vertices[j] for j in [4,5,6,7]],
         [vertices[j] for j in [0,1,5,4]],
         [vertices[j] for j in [2,3,7,6]],
         [vertices[j] for j in [1,2,6,5]],
         [vertices[j] for j in [4,7,3,0]]]
```

RESULT:

Hence, successfully implemented 3D objects using polygon modelling.